FACULTAD DE
ESTUDIOS
PROFESIONALES
ZONA MEDIA
UASLP

Medición de temperatura y control de ventilador con monitoreo gráfico

ALUMNO
Elizabeth González Ortiz
a384870@alumnos.uaslp.mx

Docente: Ing. Jesús Padrón
Fecha: 13 de marzo de 2026

***Resumen*—En esta práctica se implementó un sistema de medición de temperatura utilizando el sensor analógico LM35 conectado al convertidor analógico-digital de la Raspberry Pi Pico W. Los valores leídos se convirtieron a temperatura en grados Celsius y se enviaron por el puerto serie a una computadora, donde un script de Python con matplotlib generó una gráfica en tiempo real. Además, se incorporó un ventilador de computadora controlado mediante un transistor 2N2222, que se activa automáticamente cuando la temperatura supera los 70 °C, simulando un sistema de enfriamiento inteligente. Se analizaron conceptos fundamentales como resolución del ADC, acondicionamiento de señales y comunicación serie.**

-A Introducción

En esta práctica se desarrolló un sistema de medición de temperatura con el sensor LM35 y la Raspberry Pi Pico W, utilizando su ADC de 12 bits para convertir la señal analógica en temperatura. Los datos se enviaron por USB a una PC, donde un programa en Python generó gráficas en tiempo real con matplotlib. También se implementó el control de un ventilador mediante un transistor 2N2222A, activándolo al superar un umbral térmico para simular un sistema de enfriamiento inteligente.

Cabe destacar la colaboración de mi compañera Paola, quien apoyó en la programación debido a fallas en mi laptop que impidieron el correcto funcionamiento del SDK. A pesar de ello, trabajamos en conjunto y distribuimos las tareas del proyecto, aunque siempre nos apoyamos mutuamente en todo lo necesario.

I. DESARROLLO EXPERIMENTAL

I-A Materiales utilizados

- 1 Raspberry Pi Pico W
- 1 sensor de temperatura LM35
- 1 transistor NPN 2N2222A
- 1 ventilador de computadora (5 V)
- 1 resistencia de 1 kΩ (para la base del transistor)
- 1 protoboard
- Cables de conexión calibre 22
- Cable USB para alimentación y comunicación

I-B Configuración del entorno de programación

Para llevar a cabo la práctica se utilizó Visual Studio Code. Los pasos seguidos fueron:

1. Crear una carpeta para el proyecto, por ejemplo `Practica_11`.
2. Dentro de ella, disponer los archivos necesarios: el codigo para la grafica, y la carpeta con las configuraciones adecuadas.
3. Compilar el código y transferirlo a la placa.

I-C Script de visualización en Python

Para la visualización y registro de los datos en la computadora, se desarrolló un script en Python que utiliza las bibliotecas serial, matplotlib, tkinter y threading. Este programa permite:

- Conectarse al puerto serie donde se encuentra la Raspberry Pi Pico W.
- Leer en tiempo real las líneas enviadas con el formato `Temperatura: XX.X C`.
- Almacenar las muestras en una cola y mostrarlas gráficamente mediante matplotlib incrustado en una interfaz tkinter.

A continuación se presenta el código completo del script utilizado:

Listing 1: Script de visualización en Python (graficatemperatura.py)

```
"""
Grafica de Temperatura en Tiempo Real - PC
Lee datos del Raspberry Pi Pico via USB Serial
"""

import serial
import serial.tools.list_ports
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.backends.backend_tkagg import
    FigureCanvasTkAgg
```

```python
from collections import deque
import tkinter as tk
from tkinter import messagebox
import threading
import time
import re

#        CONFIGURACION


PUERTO = 'COM10'
BAUDRATE = 115200
MAX_PUNTOS = 100
UMBRAL_VENTILADOR = 70

#        DATOS


tiempos = deque(maxlen=MAX_PUNTOS)
temperaturas = deque(maxlen=MAX_PUNTOS)
muestras_guardadas = []
tiempo_inicio = time.time()
serial_conn = None
leyendo = False


def leer_serial():
    global serial_conn, leyendo
    while leyendo:
        try:
            if serial_conn and serial_conn.
                in_waiting:
                linea = serial_conn.readline()
                    .decode('utf-8').strip()
                match = re.search(r"
                    Temperatura:\s*([\d.]+)",
                    linea)
                if match:
                    temp = float(match.group
                        (1))
                    t = time.time() -
                        tiempo_inicio
                    tiempos.append(t)
                    temperaturas.append(temp)
                    muestras_guardadas.append
                        ((t, temp))
        except Exception:
            pass
        time.sleep(0.05)


def guardar_datos():
    if not muestras_guardadas:
        messagebox.showinfo("Sin datos", "No
            hay datos para guardar.")
        return
    with open("temperaturas.csv", "w") as f:
        f.write("Tiempo(s),Temperatura(C)\n")
        for t, temp in muestras_guardadas:
            f.write(f"{t:.2f},{temp:.1f}\n")
    messagebox.showinfo("Guardado", f"Se
        guardaron {len(muestras_guardadas)}
        muestras en 'temperaturas.csv'")


def conectar():
    global serial_conn, leyendo, tiempo_inicio
    puerto = entry_puerto.get().strip()
    try:
        serial_conn = serial.Serial(puerto,
            BAUDRATE, timeout=1)
        tiempo_inicio = time.time()
        leyendo = True
        hilo = threading.Thread(target=
            leer_serial, daemon=True)
        hilo.start()
        btn_conectar.config(state=tk.DISABLED)
        btn_desconectar.config(state=tk.NORMAL
            )
        lbl_estado.config(text=f"Conectado en
            {puerto}", fg="#a6e3a1")
    except Exception as e:
        messagebox.showerror("Error", f"No se
            pudo conectar:\n{e}")


def desconectar():
    global leyendo, serial_conn
    leyendo = False
    if serial_conn:
        serial_conn.close()
        serial_conn = None
    btn_conectar.config(state=tk.NORMAL)
    btn_desconectar.config(state=tk.DISABLED)
    lbl_estado.config(text="Desconectado", fg=
        "#f38ba8")


#        VENTANA TKINTER


root = tk.Tk()
root.title("Monitor de Temperatura LM35 -
    Raspberry Pi Pico")
root.configure(bg="#b6b6fa")
root.geometry("900x600")

frame_top = tk.Frame(root, bg="#1e1e2e", pady
    =8)
frame_top.pack(fill=tk.X)

tk.Label(frame_top, text="Monitor de
    Temperatura LM35",
         font=("Segoe UI", 16, "bold"), fg="#
             cdd6f4", bg="#1e1e2e").pack(side=
             tk.LEFT, padx=15)

lbl_temp_actual = tk.Label(frame_top, text="--
    C",
                           font=("Segoe UI",
                               22, "bold"),
                               fg="#a6e3a1",
                               bg="#1e1e2e")
lbl_temp_actual.pack(side=tk.RIGHT, padx=20)

lbl_fan = tk.Label(frame_top, text="Ventilador
    : OFF",
                   font=("Segoe UI", 11), fg="
                       #89dceb", bg="#1e1e2e")
lbl_fan.pack(side=tk.RIGHT, padx=10)

frame_ctrl = tk.Frame(root, bg="#313244", pady
```

```
    =6)
frame_ctrl.pack(fill=tk.X)

tk.Label(frame_ctrl, text="Puerto:", fg="#
    cdd6f4", bg="#313244",
          font=("Segoe␣UI", 10)).pack(side=tk.
              LEFT, padx=(15, 4))

entry_puerto = tk.Entry(frame_ctrl, width=10,
    font=("Segoe␣UI", 10),
                         bg="#45475a", fg="#
                             cdd6f4",
                             insertbackground="
                             white")
entry_puerto.insert(0, PUERTO)
entry_puerto.pack(side=tk.LEFT, padx=4)

btn_conectar = tk.Button(frame_ctrl, text="
    Conectar", command=conectar,
                          bg="#a6e3a1", fg="#1
                              e1e2e", font=("
                              Segoe␣UI", 10, "
                              bold"),
                          relief=tk.FLAT, padx
                              =10)
btn_conectar.pack(side=tk.LEFT, padx=6)

btn_desconectar = tk.Button(frame_ctrl, text="
    Desconectar", command=desconectar,
                              bg="#f38ba8", fg=
                                  "#1e1e2e",
                                  font=("Segoe␣
                                  UI", 10, "
                                  bold"),
                              relief=tk.FLAT,
                                  padx=10,
                                  state=tk.
                                  DISABLED)
btn_desconectar.pack(side=tk.LEFT, padx=4)

tk.Button(frame_ctrl, text="Guardar␣CSV",
    command=guardar_datos,
           bg="#89b4fa", fg="#1e1e2e", font=("
               Segoe␣UI", 10, "bold"),
           relief=tk.FLAT, padx=10).pack(side=
               tk.LEFT, padx=6)

lbl_estado = tk.Label(frame_ctrl, text="
    Desconectado", fg="#f38ba8",
                       bg="#313244", font=("
                           Segoe␣UI", 10))
lbl_estado.pack(side=tk.RIGHT, padx=15)

#         GRAFICA MATPLOTLIB


fig, ax = plt.subplots(figsize=(9, 4))
fig.patch.set_facecolor("#1e1e2e")
ax.set_facecolor("#181825")
ax.tick_params(colors="#cdd6f4")
ax.spines[:].set_color("#45475a")
ax.set_xlabel("Tiempo␣(s)", color="#cdd6f4",
    fontsize=10)
ax.set_ylabel("Temperatura␣(C)", color="#
    cdd6f4", fontsize=10)
ax.set_title("Temperatura␣en␣Tiempo␣Real",
    color="#cdd6f4", fontsize=12, fontweight="
    bold")
ax.grid(True, color="#313244", linestyle="--",
     alpha=0.7)

linea, = ax.plot([], [], color="#1991e7",
    linewidth=2, label="Temperatura")
ax.axhline(y=UMBRAL_VENTILADOR, color="#f38ba8
    ", linewidth=1.2,
           linestyle="--", label=f"Umbral␣
               ventilador␣({UMBRAL_VENTILADOR}
               C)")
ax.legend(loc="upper␣left", facecolor="#500066
    ", labelcolor="#cdd6f4")

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill=tk.BOTH,
    expand=True, padx=10, pady=6)


def actualizar_grafica(frame):
    if len(tiempos) > 1:
        linea.set_data(list(tiempos), list(
            temperaturas))
        ax.set_xlim(max(0, tiempos[-1] -
            MAX_PUNTOS), tiempos[-1] + 2)
        temp_min = min(temperaturas) - 5
        temp_max = max(temperaturas) + 5
        ax.set_ylim(temp_min, max(temp_max,
            UMBRAL_VENTILADOR + 10))

        ultima_temp = temperaturas[-1]
        lbl_temp_actual.config(text=f"{
            ultima_temp:.1f}␣C")

        if ultima_temp > UMBRAL_VENTILADOR:
            lbl_temp_actual.config(fg="#f38ba8
                ")
            lbl_fan.config(text="Ventilador:␣
                ON", fg="#f38ba8")
        elif ultima_temp > 50:
            lbl_temp_actual.config(fg="#fab387
                ")
            lbl_fan.config(text="Ventilador:␣
                OFF", fg="#fab387")
        else:
            lbl_temp_actual.config(fg="#a6e3a1
                ")
            lbl_fan.config(text="Ventilador:␣
                OFF", fg="#89dceb")

    canvas.draw()


ani = animation.FuncAnimation(fig,
    actualizar_grafica, interval=500,
    cache_frame_data=False)

root.mainloop()
```

I-D Carga y ejecución del programa

Para compilar y cargar el programa en la Raspberry Pi Pico W:

1. En Visual Studio Code, abrir la carpeta del proyecto y asegurarse de que el archivo esté correctamente

configurado.

2. Conectar la Pico W al PC manteniendo presionado el botón BOOTSEL para ingresar el nuevo codigo. Aparecerá como una unidad de almacenamiento.
3. La placa se reiniciará automáticamente y comenzará a ejecutar el programa.
4. Ejecutar el codigo en visual para que se muestre la grafica .

El anterior script se ejecuta en la computadora una vez que la Raspberry Pi Pico W está transmitiendo datos. La interfaz gráfica permite una interacción sencilla: seleccionar el puerto COM, conectar y observar la gráfica en tiempo real.

II. RESULTADOS

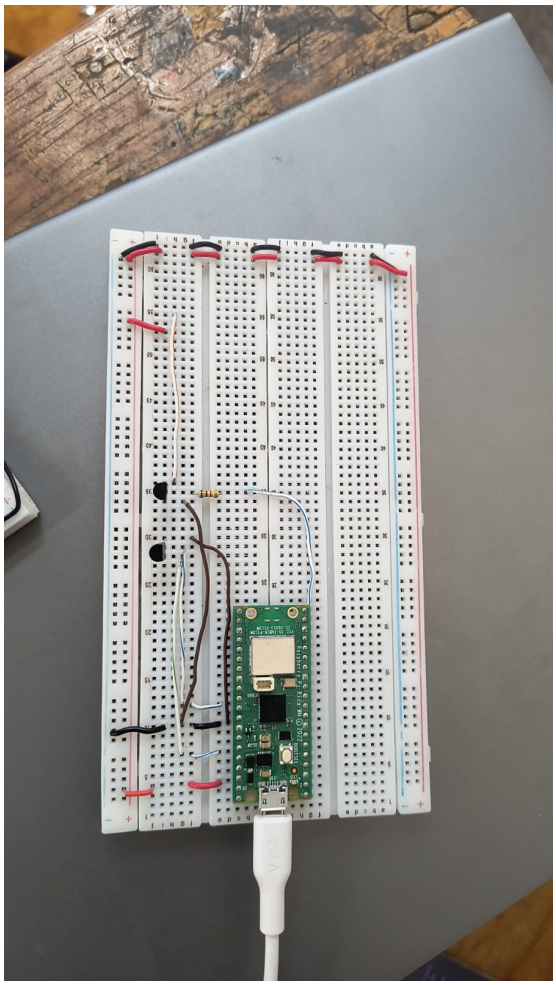

Figura 1: Fotografía del circuito armado en protoboard

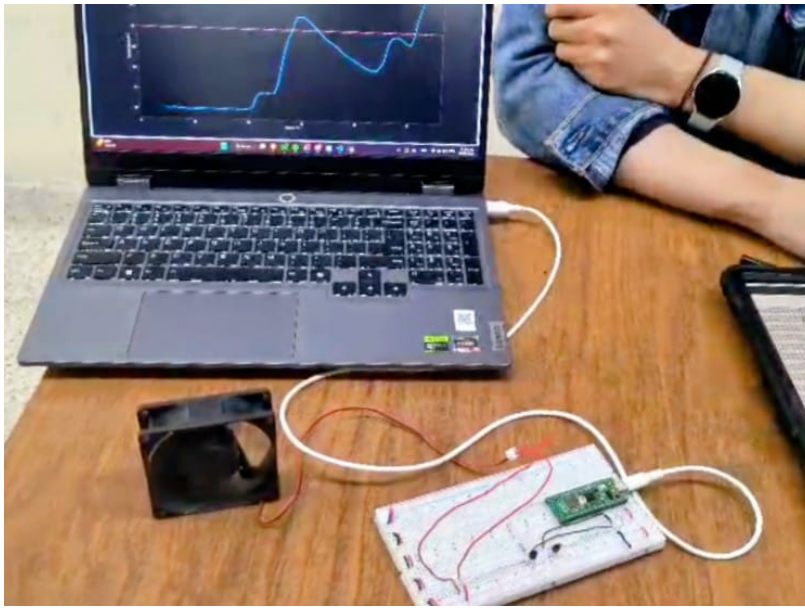

Figura 2: Fotografía del circuito en funcionamiento

II-A Armado del circuito

El circuito se construyó siguiendo las indicaciones de la práctica:

- El pin VCC del LM35 se conectó a 3.3 V de la Pico.
- El pin GND del LM35 se conectó a GND común.
- El pin OUT del LM35 se conectó al GP26.
- Para el control del ventilador, se utilizó un transistor 2N2222A: el colector al ventilador, el emisor a GND y la base al GP15 a través de una resistencia de 1k Ω.

II-B Problemas encontrados

- La temperatura leída era incorrecta.
- El ventilador no se activaba al superar los 70°C. **S**e midió el voltaje en la base del transistor, resultó que el pin GP15 no estaba configurado correctamente como salida.

II-C Comportamiento observado

Una vez resueltos los problemas, el sistema funcionó correctamente. Al acercar una fuente de calor al LM35, se observó el aumento gradual de la temperatura en el monitor serie.

III. CONCLUSIONES

El sistema implementado funcionó correctamente, logrando medir la temperatura ambiente mediante el sensor LM35 y visualizarla en el monitor serie de forma continua. Se realizaron pruebas acercando una fuente de calor al sensor observando una respuesta idonea para la practica.

El control del ventilador mediante el transistor 2N2222A respondió de manera eficaz al umbral establecido, encendiéndose y apagándose automáticamente según la temperatura medida.

Además, la integración con Python y matplotlib permitió generar gráficas en tiempo real, facilitando el análisis del comportamiento térmico. Gracias a la ayuda de mi companera Paola este proyecto fue un poco mas sencillo. ademas se trabajoo en companerismo porque mi computadora tiene cada vez mas problemas que estot tratando de corregir

IV. PREGUNTAS DE REPASO

1. **¿Qué es un convertidor analógico-digital (ADC)?** Es un dispositivo electrónico que convierte una señal analógica continua, como un voltaje, en un valor digital discreto proporcional a la magnitud de la señal. Permite que los microcontroladores procesen señales del mundo real.
2. **¿Qué resolución tiene el ADC del Raspberry Pi Pico W?** El ADC de la Raspberry Pi Pico W tiene una resolución de 12 bits, lo que significa que puede digitalizar la señal en 4096 niveles distintos (0 a 4095).
3. **¿Por qué es necesario convertir el valor digital a voltaje?** Porque la lectura digital es solo un número sin unidades. Convertir a voltaje permite relacionar ese número con la magnitud física real que se está midiendo, utilizando el rango de referencia del ADC generalmente 0.1–3.3 V que es lo que soporta el microcontrolador.
4. **¿Cómo se obtiene la temperatura a partir del voltaje del sensor LM35?** El LM35 entrega 10 mV por cada grado Celsius, por lo que la temperatura en °C es igual al voltaje medido en milivoltios dividido entre 10.
5. **¿Qué sucede si aumenta la resolución del ADC?** Aumenta la precisión de la medición, ya que se tienen más niveles para representar el mismo rango de voltaje. Esto reduce el error de cuantización y permite detectar variaciones más pequeñas en la señal de entrada.
6. **¿Por qué es útil representar los datos mediante una gráfica?** La gráfica permite visualizar tendencias, picos, estabilidad y comportamientos transitorios que no son evidentes con lecturas puntuales. Facilita el análisis y la toma de decisiones en sistemas de monitoreo.
7. **¿Qué aplicaciones industriales tiene la medición de temperatura con microcontroladores?** Control de procesos químicos, monitoreo de motores y maquinaria, sistemas HVAC, hornos industriales, alarmas de sobrecfalentamiento, y estaciones meteorológicas, entre otros.
8. **¿Qué ventajas tiene registrar datos en el tiempo en lugar de observar solo un valor instantáneo?** Permite detectar cambios graduales, calcular derivadas, identificar patrones cíclicos, llevar un historial para mantenimiento predictivo y realizar análisis estadísticos.

Referencias

[1] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet: An RP2040-based microcontroller board with wireless*, 2023. [En línea]. Disponible: https://components101.com/sites/default/files/component$_{d}atasheet/pi-pico-w-datasheet.pdf$